

Helix Thready — Database Migration Strategy

Table of Contents

1. Engine: `digital.vasic.database/pkg/migration`
2. The Runner contract (VERIFIED from source)
3. Loading `.sql` files into `migration.Migration`
4. Wiring the runner at boot
5. Expand / Contract (zero-downtime)
6. Rollback
7. Concurrency: advisory lock
8. Verified caveats & required fixes (anti-bluff)
9. Migration roadmap (0001–0007)
10. CI-less enforcement & TDD
11. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (database)	Initial: migration.Runner contract, expand-contract, loader, rollback, CI-less enforcement, verified caveats
2	2026-07-22	swarm (database, Pass 3)	Shipped runnable <code>0002</code> – <code>0007</code> migration files (all six previously roadmap-only); reconciled the roadmap table to actual file contents; documented the non-transactional apply path for <code>0007</code> and the full-list bootstrap-vs-online index recipe
3	2026-07-22	swarm (docs export)	Fixed inline mermaid syntax so diagram renders

Table of Contents

1. Engine: `digital.vasic.database/pkg/migration`
2. The Runner contract (VERIFIED from source)
3. Loading `.sql` files into `migration.Migration`
4. Wiring the runner at boot

5. Expand / Contract (zero-downtime)
6. Rollback
7. Concurrency: advisory lock
8. Verified caveats & required fixes (anti-bluff)
9. Migration roadmap (0001–0007)
10. CI-less enforcement & TDD
11. Open items

1. Engine: `digital.vasic.database/pkg/migration`

All schema changes are automated, versioned migrations applied through the in-house `digital.vasic.database` migration engine — no external tool (not golang-migrate, not Flyway, not Atlas). `[IN-HOUSE: database]` `[final request Q30, $2.1.1]` Development uses SQLite (`modernc.org/sqlite`, cgo-free); production uses PostgreSQL (`pgx/v5`); the driver is selected by `database.Config.Driver` (`"sqlite" | "postgres"`, VERIFIED in `pkg/database/database.go`). The same migration list runs against both, so dev parity is real — subject to the dialect caveat in §8.

2. The Runner contract (VERIFIED from source)

The following is the exact public surface of `pkg/migration` (read at source — `digital.vasic.database/pkg/migration/migration.go`), not an approximation:

```
// A single migration. Up applies; Down reverses.
type Migration struct {
    Version int    // unique, monotonically increasing
    Name     string // human-readable
    Up       string // SQL to apply
    Down     string // SQL to reverse
}

// Runner applies/rolls back migrations against a db.Database.
func NewRunner(database db.Database, table string) *Runner // table default: "schema_migrations"

func (r *Runner) Init(ctx) error // CREATE TABLE IF NOT EXISTS
                             <table>

func (r *Runner) Applied(ctx) ([]int, error) // applied versions, ascending
func (r *Runner) Apply(ctx, migrations []Migration) error // Init + apply all pending,
                             in version order
func (r *Runner) Rollback(ctx, version int) error // returns error: "use
                             RollbackWith"
func (r *Runner) RollbackWith(ctx, version int, migs []Migration) error // reverse
                             version>=target, descending
```

Semantics (VERIFIED):

- `Init` creates the tracking table: `CREATE TABLE IF NOT EXISTS schema_migrations (version INTEGER PRIMARY KEY, name TEXT NOT NULL, applied_at TIMESTAMP NOT NULL)`.
- `Apply` calls `Init`, reads `Applied`, sorts the supplied migrations by `Version`, and applies only those not yet recorded. Each migration is applied by `applyOne`, which is **transactional**: `Begin` → `tx.Exec(m.Up)` → `INSERT` the tracking row → `Commit`; any error triggers `tx.Rollback`. So **one migration = one atomic transaction** — a failed `Up` leaves the DB untouched and unrecorded.
- `Rollback(version)` is intentionally inert: it returns an error instructing you to call `RollbackWith`, because the plain method has no access to `Down` SQL. **Always use `RollbackWith`**.
- `RollbackWith` reverses every applied migration with `Version >= target` in **descending** order, each in its own transaction, and errors if a migration lacks `Down` SQL.

Consequences for how we author migrations:

- Because `Up` runs in a single transaction, **`CREATE INDEX CONCURRENTLY` and other non-transactional statements cannot appear in a runner-managed `Up`** — see §8.
- Because `applyOne` sends `m.Up` with **no bind arguments**, pgx uses the *simple query protocol*, which permits **multiple statements in one `Up`** (our `0001_init` relies on this). Never parameterise migration bodies — inline literal values.

3. Loading `.sql` files into `migration.Migration`

We keep migrations as reviewable `.sql` files (`migrations/0001_init.sql`) and load them into `[]migration.Migration` with a tiny parser that splits on the `-- +thready Up` / `-- +thready Down` markers. This keeps SQL diff-friendly while feeding the Go runner.

```
// loader.go — parse migrations/NNNN_name.sql into migration.Migration values.
var reFile = regexp.MustCompile(`^(\\d{4})_([a-z0-9_]+)\\.sql$`)

func Load(fsys fs.FS, dir string) ([]migration.Migration, error) {
    entries, err := fs.ReadDir(fsys, dir)
    if err != nil { return nil, err }
    var out []migration.Migration
    for _, e := range entries {
        m := reFile.FindStringSubmatch(e.Name())
        if m == nil { continue }
        version, _ := strconv.Atoi(m[1])
        body, err := fs.ReadFile(fsys, path.Join(dir, e.Name()))
        if err != nil { return nil, err }
        up, down := splitMarkers(string(body)) // on "-- +thready Up" / "-- +thready Down"
        out = append(out, migration.Migration{
            Version: version, Name: m[2], Up: up, Down: down,
        })
    }
    sort.Slice(out, func(i, j int) bool { return out[i].Version < out[j].Version })
    return out, nil
}
```

Migrations are embedded with `//go:embed migrations/*.sql` so the binary is self-contained (no runtime file dependency) — matching the org’s single-artifact deployment.

4. Wiring the runner at boot

```
//go:embed migrations/*.sql
var migrationsFS embed.FS

func Migrate(ctx context.Context, cfg database.Config) error {
    dbc := postgres.NewClient(&cfg) // or sqlite.NewClient in dev, per cfg.Driver
    if err := dbc.Connect(ctx); err != nil { return err }
    defer dbc.Close()

    migs, err := Load(migrationsFS, "migrations")
    if err != nil { return err }

    runner := migration.NewRunner(dbc, "schema_migrations")

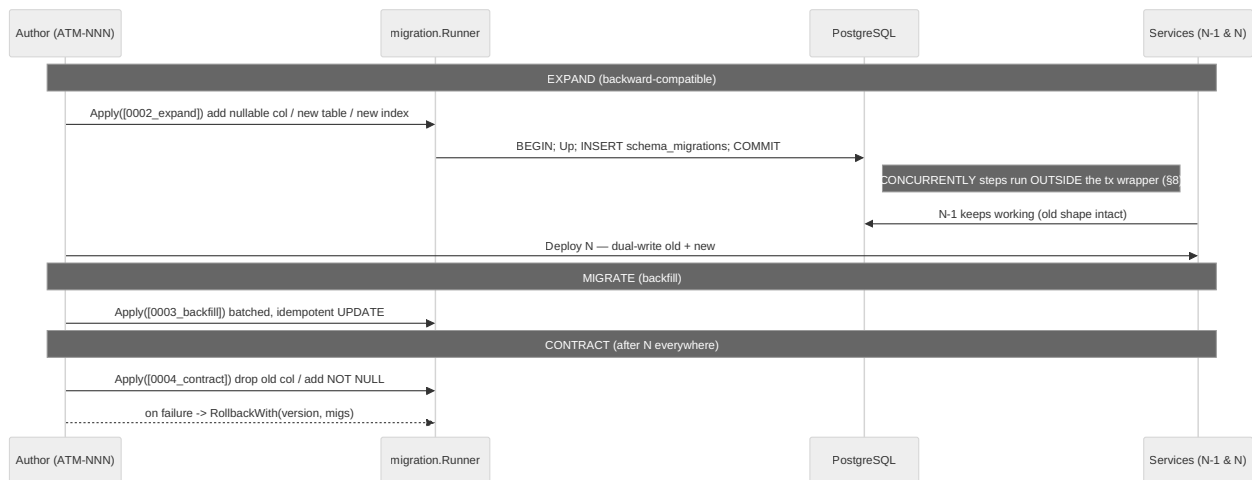
    // Single-flight across replicas/deploy: hold an advisory lock (see §7).
    return withAdvisoryLock(ctx, dbc, lockKeyMigrations, func() error {
        return runner.Apply(ctx, migs) // Init + apply pending, atomically per migration
    })
}
```

This runs at service start (and from the deploy script) so an environment converges to the latest schema before serving traffic. `Apply` is idempotent — already-applied versions are skipped — so re-running is safe.

5. Expand / Contract (zero-downtime)

Schema changes on the live system follow **expand** → **migrate** → **contract** so that version N-1 and N of the services run simultaneously without breakage ([final request Q30, \$22.8]).

Source: `diagrams/migration-expand-contract.mmd`.



Explanation (for readers/models that cannot see the diagram). The sequence diagram has four participants — the change **Author** (an `ATM-NNN` work item), the `migration.Runner`, **PostgreSQL**, and the running **Services** (both version N-1 and N during a rollout) — and walks them through the three phases separated by the `Note over` bands.

In **expand**, the author applies a migration (here labelled `0002_expand`) that adds only backward-compatible things — a nullable column, a new table, or a new index — so the still-running N-1 services are unaffected; the new column/table is invisible to them. The runner applies it inside its `BEGIN; Up; INSERT schema_migrations; COMMIT` transaction and records the version, and the diagram's side-note flags that any **CONCURRENTLY** step must run *outside* that transaction wrapper (§8.2). Version N of the services is then deployed and **dual-writes** the old and new shapes so both readers stay correct while the fleet is mixed.

In **migrate**, a separate migration backfills existing rows in idempotent batches — safe to re-run and chunked to avoid long locks. Only once N is deployed everywhere and nothing reads the old shape does **contract** remove it: dropping the old column or tightening a constraint to `NOT NULL`. Each phase is its own reviewable migration, and the final dashed edge shows the failure path — `RollbackWith(version, migs)` reverses a phase using its `Down` SQL. This three-phase discipline is why our enum-like columns are `TEXT + CHECK` (widening a CHECK is an expand-only `ALTER`, not a type rewrite) and why the firehose tables avoid incoming FKs (adding or removing them online is costly).

Concrete example — widening a category domain without a rewrite:

```
-- 0005_expand_category_precedence.sql (EXPAND only; no contract needed)
-- +thready Up
ALTER TABLE categories DROP CONSTRAINT categories_precedence_class_check;
ALTER TABLE categories ADD CONSTRAINT categories_precedence_class_check
    CHECK (precedence_class IN ('download','convert','analyze','research','reply','translate'));
-- +thready Down
ALTER TABLE categories DROP CONSTRAINT categories_precedence_class_check;
ALTER TABLE categories ADD CONSTRAINT categories_precedence_class_check
    CHECK (precedence_class IN ('download','convert','analyze','research','reply'));
```

6. Rollback

- Use `RollbackWith(ctx, version, migs)` (never the inert `Rollback`). It reverses every applied migration `>= version` in descending order, each in its own transaction.
- Every migration MUST ship a real `Down`. `RollbackWith` errors loudly if a `Down` is empty — that is intentional; a migration without a reverse is a release blocker.
- Some operations are **not reversible by data** (a `DROP COLUMN` in contract loses data). For those, the `Down` recreates the *structure* but the *data* is recovered from backup (Q41 PITR) — the runbook documents this, and the contract migration is gated on a fresh backup.
- Forward-fix is preferred over rollback for already-released schema: author `NNNN+1` rather than rolling back a shipped version, matching the no-force-push release rule `[CONSTITUTION §11.4.113]`.

7. Concurrency: advisory lock

VERIFIED: the runner takes **no lock**; `Apply` reads `Applied` then applies pending migrations without guarding against a second process doing the same. With multiple service replicas booting at once (or a boot racing the deploy script), two runners could attempt the same migration. We wrap `Apply` in a PostgreSQL **advisory lock** so exactly one runner migrates at a time:

```
func withAdvisoryLock(ctx context.Context, dbc database.Database, key int64, fn func() error)
    error {
    if _, err := dbc.Exec(ctx, "SELECT pg_advisory_lock($1)", key); err != nil { return err }
    defer func() { _, _ = dbc.Exec(ctx, "SELECT pg_advisory_unlock($1)", key) }()
    return fn() // only one process holds the lock; others block then no-op (already applied)
}
```

The same lock discipline guards the partition-maintenance job's DDL (see [partitioning.md §5](#)). [\[GAP: session_orchestrator-2.9\]](#) the design-only atomic-claim registry is not needed here — Postgres advisory locks provide the single-flight primitive directly.

8. Verified caveats & required fixes (anti-bluff)

These are read-at-source findings about the migration engine. They are **not** cosmetic — they gate correctness on the Postgres path and **MUST** be handled before relying on the runner in production. None of them is papered over.

1. [\[OPEN: migration-runner-pg-placeholders\]](#) — **the runner's tracking-table writes use ? placeholders (SQLite-native), which pgx does not translate.** VERIFIED: `applyOne` emits `INSERT INTO <table> (version, name, applied_at) VALUES (?, ?, ?)` and `rollbackOne` emits `DELETE FROM <table> WHERE version = ?`, executed via the `db.Tx`. The Postgres transport (`postgres.Client.Exec` / `pgTx.Exec`) passes the query to pgx **unmodified** — it does **not** call the dialect's `RewritePlaceholders` (that rewriter lives in the separate `database/sql`-based `connection.DB`, which does **not** implement the pgx `db.Database` interface the runner needs). On PostgreSQL, pgx expects `$1,$2,$3`, so these bookkeeping writes fail. The user's `Up` / `Down` DDL is **unaffected** (it has no bind args). **Plan (ATM-DB-001):** either (a) inject a thin dialect-aware `db.Database` decorator that rewrites `? → $n` for the runner, or (b) submit a one-line upstream fix so the runner emits driver-appropriate placeholders (preferred; folds back to `digital.vasic.database`). Until closed, a paired **anti-bluff test** (§10) runs the runner against a real Postgres container and asserts the tracking row is written — a green unit test on SQLite alone would be a false pass.
2. [\[OPEN: migration-concurrently\]](#) — **CREATE INDEX CONCURRENTLY cannot run inside the runner's transaction.** VERIFIED: `applyOne` wraps `Up` in `Begin/Commit`; Postgres forbids `CONCURRENTLY` in a transaction block. **Plan (ATM-DB-002):** the secondary-index migration (`0007`) is applied by a **non-transactional path** — a dedicated runner mode or a deploy-step that executes each `CREATE INDEX CONCURRENTLY` on its own autocommit connection and records the version manually — kept separate from the DDL migrations that the transactional runner handles. `0001`'s indexes are on empty tables, so plain `CREATE INDEX` there is fine.
3. **Tracking-table timestamp type.** `Init` declares `applied_at` `TIMESTAMP` (not `TIMESTAMPZ`) and inserts `time.Now().UTC()`. Cosmetic on a UTC server, but noted so the audit of “when was this applied” is read as UTC. No action required.

Per the quality covenant, these are surfaced, not hidden: the migration engine is PRODUCTION for SQLite and its DDL path, but its Postgres bookkeeping path is treated as **FLAGGED until the paired container test is green**. [\[final request §4.3\]](#)

9. Migration roadmap (0001–0007)

All seven migrations are now shipped as reviewable `.sql` files under `migrations/` (Pass 3 added `0002` – `0007` ; `0001` shipped in Wave 1). Each mirrors the corresponding domain in `schema-relational.sql` / `schema-vector.sql` exactly.

Version	File	Phase	Transactional?	Contents
0001	<code>0001_init.sql</code>	expand	yes	accounts, users, roles, permissions, role_permissions, memberships, messengers, messenger_accounts, channels, threads, posts (partitioned), replies (partitioned), processing_state + hot claim index
0002	<code>0002_classification.sql</code>	expand	yes	hashtags, categories, hashtag_categories, post_hashtags, reply_hashtags, post_categories
0003	<code>0003_assets.sql</code>	expand	yes	skills, skill_runs, generated_artifacts, assets, asset_links (skills/artifacts precede assets because <code>asset_links</code> has a real FK into <code>generated_artifacts</code>)
0004	<code>0004_billing.sql</code>	expand	yes	plans, subscriptions, usage_records, invoices
0005	<code>0005_events_audit.sql</code>	expand	yes	events (partitioned), event_subscriptions, audit_log (partitioned), archived_partitions
0006	<code>0006_vector_collections.sql</code>	expand	yes	vectordb_posts/ replies/assets/ generated (tables only — matches the adapter's DDL; no ANN index here)
0007		expand	no (CONCURRENTLY)	all relational secondary indexes + FTS (<code>body_fts</code>) +

Version	File	Phase	Transactional?	Contents
	0007_secondary_index.es.sql			GIN) + vector ANN (HNSW) + metadata GIN; applied by the non-transactional deploy step (§8.2)

Why the split is exactly this. 0001 – 0006 are pure DDL with no bind parameters, so each runs cleanly inside the runner’s single `applyOne` transaction (multi-statement `Up` via the pgx simple protocol). 0007 is isolated precisely because `CREATE INDEX CONCURRENTLY` cannot run in a transaction (§8.2 / `ATM-DB-002`); it carries **every** index that is not a structural PK/UNIQUE (those ship inline in 0001 – 0006) plus the FTS generated column. The 0007 header documents the bootstrap-vs-online distinction for partitioned parents (plain `CREATE INDEX` on empty parents at init; the ON ONLY + per-partition `ATTACH` recipe from `indexing.md §7` for a populated deployment).

Small, single-purpose migrations keep each one reviewable by the independent AI review gate (Fable @ xhigh) and cheap to roll back. Partition-maintenance DDL (new monthly partitions) is **not** a migration — it is the scheduled job in `partitioning.md`. Every migration ships a real `Down`; the from-empty apply-then-rollback of the whole list on both drivers is the pre-tag gate in §10.

10. CI-less enforcement & TDD

There is **no server-side CI** [`CONSTITUTION §11.4.156`]. Migration safety is enforced locally and at release:

- **Local git-hook** runs the DB test bank (`go test -race ./...`) before commit; migrations must apply **and** roll back cleanly on a real Postgres + SQLite container.
- **Pre-tag full-suite retest** [`§11.4.40`] re-applies the entire migration list from empty on both drivers and asserts the resulting schema matches `schema-relational.sql` (structural diff) before a `<PREFIX>-<ver>` tag; push to all four upstreams [`§2.1`].
- **TDD reproduce-first** [`§11.4.43`] — the RED test for every migration:

```
// RED first: 0001 must apply and roll back cleanly on a REAL Postgres (anti-bluff, closes ATM-DB-001).
func TestMigration0001_ApplyRollback_Postgres(t *testing.T) {
    dbc := realPostgres(t) // testcontainers; SKIP-OK if no engine (CONST-035)
    runner := migration.NewRunner(dbc, "schema_migrations")
    migs := mustLoad(t, "migrations")

    require.NoError(t, runner.Apply(ctx, migs[:1])) // apply 0001
    require.ElementsMatch(t, []int{1}, mustApplied(t, runner)) // tracking row written on PG
    // proves ATM-DB-001 fixed
    require.True(t, tableExists(t, dbc, "posts_2026_07")) // partition created

    require.NoError(t, runner.RollbackWith(ctx, 1, migs[:1])) // down
    require.Empty(t, mustApplied(t, runner))
    require.False(t, tableExists(t, dbc, "accounts"))
}

// Idempotency: re-Apply is a no-op.
func TestMigration_ApplyTwice_NoOp(t *testing.T) {
    // ... Apply, Apply again -> second returns nil, Applied() unchanged.
}
```

The Postgres variant of `TestMigration0001_ApplyRollback_Postgres` is the paired mutation / anti-bluff gate for caveat §8.1: a green run on SQLite alone does **not** satisfy it.

11. Open items

ID	Item	Plan
ATM-DB-001	[OPEN: migration-runner-pg-placeholders] ? vs \$n on pgx	Dialect decorator or upstream fix; guarded by real-PG test (§8.1)
ATM-DB-002	[OPEN: migration-concurrently] CONCURRENTLY outside runner tx	Non-transactional path for 0007 (§8.2)
ATM-DB-004	[OPEN: db-partition-fk] app- vs DB-enforced FK	Decide per env (partitioning.md §8)

Made with love ♥ by Helix Development.